

Parallel Training for Neural MAC Polar Decoders: Research Report

The Open Problem and What We Found

1. Context

We have a neural successive cancellation (SC) decoder for two-user MAC polar codes. It replaces the analytical 2×2 probability tensor operations (CalcLeft, CalcRight, CalcParent) with weight-shared MLPs operating on d -dimensional embeddings. The decoder makes 4-class joint (u,v) decisions at each leaf of a binary computation tree.

What works: At block lengths $N \leq 128$, the neural decoder matches the analytical SC decoder within 4% BLER. With $d=32$ (153K params), it actually beats SC by 20% at $N=32$ and $N=64$.

What fails: At $N \geq 256$, training collapses. The decoder has $O(N \log N)$ sequential operations (~ 1500 at $N=256$), and gradients must flow through all of them. The optimization cannot handle this depth.

The goal: Find a way to train this decoder with $O(\log N)$ gradient depth instead of $O(N \log N)$, analogous to the “fast_ce” technique used in the single-user Neural Polar Decoder (NPD) by Aharoni et al. (2024).

2. How Fast_CE Works for Single-User NPD

In the single-user NPD, the SC tree walk has two operations at each node:

- **CheckNode (f-node):** Takes $(e_{\text{odd}}, e_{\text{even}})$ embeddings $\rightarrow$ produces left-child embedding
- **BitNode (g-node):** Takes $(e_{\text{odd}} \times u_{\text{sign}}, e_{\text{even}})$ $\rightarrow$ produces right-child embedding, where $u_{\text{sign}} = (-1)^u$ for the decided bit u

During teacher-forced training, the true bit u is known. Therefore, at each tree depth d , ALL CheckNode and BitNode operations at that depth can be computed **simultaneously** — they are independent given the layer above and the true bits. This gives $O(\log N)$ sequential steps instead of $O(N \log N)$.

Why it works in practice: The BitNode has a residual connection:

$$\text{output} = \text{MLP}(e_{\text{odd}} \times u_{\text{sign}}, e_{\text{even}}) + e_{\text{odd}} \times u_{\text{sign}} + e_{\text{even}}$$

This residual is very close to the analytical formula $g(a, b, u) = (-1)^u \times a + b$. When the model makes a wrong binary decision during sequential decode, u_{sign} flips from $+1$ to -1 — the embedding is still “well-formed” (just the mirror image). The MLP only learns a small correction to the analytical formula, so this sign flip causes a bounded perturbation.

3. The MAC Problem: 4-Class Decisions

In the two-user MAC, the decoder makes joint $(u,v) \in \{(0,0), (0,1), (1,0), (1,1)\}$ decisions at each leaf — 4 classes instead of 2. The CalcLeft operation is a circular convolution over the Klein four-group $Z_2 \times Z_2$, which couples all 4 elements of the 2×2 probability tensor.

We explored two approaches to enable fast_ce for this 4-class MAC setting.

4. Approach 1: Walsh-Hadamard Transform (WHT) Decomposition

The Mathematical Insight The circular convolution over $Z_2 \times Z_2$ can be diagonalized by the Walsh-Hadamard Transform. In the WHT domain, CalcLeft becomes **element-wise multiplication**:

$$\text{WHT}(P_{\text{left}}) = \text{WHT}(P_{\text{parent}}) \quad \text{WHT}(P_{\text{right}})$$

where $\square$ denotes element-wise product. The 4 WHT coefficients are completely independent.

The WHT matrix for $Z_{\square} \times Z_{\square}$ is:

$$\text{WHT} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

In the WHT domain: - CalcLeft = element-wise multiplication (4 independent scalar channels) - CalcRight = element-wise multiplication with character-dependent signs

The 4 characters of $Z_{\square} \times Z_{\square}$ give sign patterns for the BitNode:

Channel 0: $(u, v) = 1 \rightarrow$ no sign flip
Channel 1: $(u, v) = (-1)^v \rightarrow$ flip by v
Channel 2: $(u, v) = (-1)^u \rightarrow$ flip by u
Channel 3: $(u, v) = (-1)^{(u \oplus v)} \rightarrow$ flip by $u \oplus v$

What We Built A joint MAC decoder operating in WHT domain: - $z_encoder$ maps channel output z to $K=4$ WHT-domain channels, each with d dimensions - Per-channel CheckNode and BitNode (shared or separate MLPs) - Classifier combines 4 channel LLRs $\rightarrow$ 4-class joint decision - $Fast_ce$ processes all positions at each depth in parallel

Implementation Details We Had to Debug Three critical bugs were found and fixed during implementation:

1. **Bit-reversal mapping:** The NPD's recursive tree visits leaves in bit-reversed order relative to standard polar codes. Frozen sets and channel outputs must be permuted by $bit_reversal_perm(n)$.
2. **Codeword reconstruction in decode:** The sequential decode must return the **codeword** of each subtree (via butterfly: $x[0::2] = x_left \square x_right$, $x[1::2] = x_right$), not the message bits. The parent's BitNode needs the codeword, not the message.
3. **Sign convention:** The BitNode uses $u_sign = 2u - 1$ (mapping $0 \rightarrow -1$, $1 \rightarrow +1$), matching the NPD convention.

Results We tested multiple WHT decoder variants at $N=32$, GMAC Class B, $SNR=6dB$ (SC BLER = 0.046):

Model	Params	Best BLER	vs SC
WHT shared ops, binary sign flip	6K	0.606	13.2×
WHT shared ops, character sign flips	23K	0.336	7.3×
WHT per-channel ops, large	283K	0.254	5.5×
WHT small, 10% noisy teacher forcing	5K	0.378	8.2×
WHT small, 20% noisy teacher forcing	5K	0.414	9.0×

Character-dependent sign flips (using all 4 WHT characters) improved from 13.2× to 7.3×. Per-channel MLPs improved to 5.5×. But all variants plateau around 5-8× SC.

5. Approach 2: Direct 4-Class Fast_CE (No WHT)

We then realized that fast_ce parallelizes across **positions at each tree depth**, not across the 4 tensor elements within a position. The CalcLeft coupling within each position is handled by the MLP — different positions at the same depth ARE independent.

So we implemented fast_ce with the original architecture: - $d=16$ embeddings (same as our proven tree walk decoder) - CalcLeft MLP: $(2d \rightarrow \text{hidden} \rightarrow d)$ — handles the full 4-element coupling - BitNode conditioned on the 4-class joint decision via learned embedding - emb2logits: $d \rightarrow 4\text{-class logits}$

Result BLER = **0.340**, **7.4 $\times$ SC** with 26K params. Exactly the same performance as the WHT decomposition.

This proves that **the WHT decomposition is not lossy** — the bottleneck is not CalcLeft coupling. The bottleneck is somewhere else in the fast_ce $\rightarrow$ sequential decode pipeline.

6. Analysis: Why Fast_CE Has a 7 $\times$ Ceiling for 4-Class MAC

All fast_ce variants (WHT and non-WHT) converge to the same $\sim 7\times$ gap. The fundamental issue is the **train-test distribution mismatch** in the BitNode:

During fast_ce training (teacher forcing): - BitNode receives the TRUE joint (u,v) class at each position
- All 4-class decisions are correct - The BitNode always sees “valid” conditioning inputs

During sequential decode (inference): - BitNode receives the MODEL’S PREDICTED (u,v) class - Some predictions are wrong (especially early in decoding) - Wrong 4-class predictions create conditioning inputs the model never saw during training

Why this is worse for 4-class than binary:

In **binary** (single-user NPD): A wrong bit flips u_{sign} from +1 to -1. Due to the strong residual $e_{\text{odd}} \times u_{\text{sign}} + e_{\text{even}}$, this is just a sign flip of the embedding — the output is still “well-formed” and within the training distribution. The model has effectively seen both +1 and -1 patterns equally during training (since bits are random).

In **4-class** (MAC): A wrong joint (u,v) prediction substitutes one of 3 alternative conditioning patterns. For example, if the true class is (0,0) but the model predicts (1,0): - WHT channel 0: sign unchanged (correct) - WHT channel 1: sign unchanged (correct) - WHT channel 2: sign FLIPPED (wrong) - WHT channel 3: sign FLIPPED (wrong)

This creates a conditioning pattern that is **partially correct, partially wrong** — a mix that the model never encounters during teacher-forced training. Unlike the binary case where a flip is symmetric, the 4-class case has 3 distinct error patterns, each creating a different partial corruption.

Noisy teacher forcing does not help because adding random noise during training is not the same distribution as the model’s systematic prediction errors. The model’s errors are correlated with the channel output and code structure, while random noise is independent.

7. Comparison: Sequential Training vs Fast_CE

Property	Sequential Tree Walk	Fast_CE
Training depth	$O(N \log N)$	$O(\log N)$

Property	Sequential Tree Walk	Fast_CE
N=32 BLER	0.046 (matches SC)	0.340 (7.4× SC)
N=128 BLER	0.017 (1.04× SC)	Not tested (expected ~7× SC)
N=256 BLER	0.015 (3× SC, limited by gradient depth)	Not tested
Train-test gap	None (sequential in both)	Large (teacher forcing vs predictions)
Scaling	Fails at $N \geq 256$	$O(\log N)$ at any N

The sequential tree walk is superior in quality but cannot scale. Fast_CE scales perfectly but produces a 7× weaker decoder.

8. What We Have Not Tried

The following approaches might bridge the gap:

8.1 Scheduled Sampling for Fast_CE During fast_ce, at each depth, with probability p replace the true joint (u,v) class with the model’s own prediction. This requires a two-pass approach: 1. Forward pass with teacher forcing to get predictions 2. Second pass mixing predictions with true values

This directly addresses the train-test gap by exposing the BitNode to its own error patterns during training. The challenge is maintaining the $O(\log N)$ parallelism — the first pass is parallel, but the second pass depends on the first.

Reference: Duckworth et al. (2019), “Parallel Scheduled Sampling” — achieved 11.5% improvement in dialog generation.

8.2 Hybrid: Fast_CE Pretraining + Sequential Fine-Tuning Use fast_ce for initial training ($O(\log N)$, fast convergence to ~0.34 BLER), then fine-tune with sequential tree walk training ($O(N \log N)$, closes the gap). The fine-tuning starts from a much better initialization than random, potentially requiring far fewer sequential iterations.

For N=256: fast_ce pretraining takes minutes, then sequential fine-tuning might converge in 10K iterations instead of 100K+ (because the model starts from a good point rather than random).

We attempted this but hit an implementation issue (architecture mismatch between the fast_ce model and tree walk model). A clean implementation with shared architecture would be the natural next step.

8.3 Gradient Detaching at Intermediate Depths During sequential training, detach gradients every K steps (e.g., every $\log N$ steps). This limits gradient depth to K while still training sequentially. The detached segments don’t receive gradients, but their inputs are correct (from the sequential execution).

This is a middle ground: $O(K \times N/K) = O(N)$ total gradient flow, but maximum depth is $K = O(\log N)$.

8.4 Learned Iterative Refinement Instead of a single tree walk, do multiple passes: 1. First pass: fast_ce-trained decoder gives initial estimates (BLER ~0.34) 2. Second pass: same decoder, but BitNode receives predictions from pass 1 instead of true bits 3. Train end-to-end for 2-3 passes

Each pass is $O(\log N)$ depth. The model learns to correct its own errors from the previous pass. During training, pass 1 uses teacher forcing, pass 2 uses pass 1's predictions — this naturally exposes the model to its own error patterns.

8.5 Per-Depth Noise Calibration Instead of uniform random noise, analyze the model's actual error patterns at each tree depth and inject calibrated noise that matches those patterns. This would be a more targeted version of noisy teacher forcing.

8.6 Separate Training of CheckNode and BitNode Train the CheckNode with fast_ce (it doesn't depend on decisions, so there's no train-test gap). Then train the BitNode sequentially, but only the BitNode — with the CheckNode frozen. This limits the sequential depth to $O(N)$ (just the BitNode chain) rather than $O(N \log N)$.

8.7 Knowledge Distillation from Analytical Decoder For channels where the analytical SC decoder is available (like GMAC), use the analytical decoder's intermediate tensors as training targets. Train each tree depth independently to match the analytical intermediate results. This gives $O(1)$ depth per depth level, $O(\log N)$ total.

We tried this (“snapshot training”) and it failed because the neural embeddings live in a different space than the analytical tensors. However, combining it with a learned domain transformation (e.g., train a projection from embedding space to tensor space) might work.

9. The Fundamental Question

The core question remains:

Can we train a neural decoder for 4-class MAC polar codes with $O(\log N)$ gradient depth while achieving performance comparable to the $O(N \log N)$ sequential tree walk?

Single-user NPD answers “yes” for 2-class. Our experiments show “not yet” for 4-class. The $7\times$ gap appears to be fundamental to the teacher-forcing approach with 4-class decisions.

Possible resolutions: 1. Find a training method that doesn't rely on teacher forcing (reinforcement learning, evolutionary strategies) 2. Find a way to make the 4-class BitNode as robust to prediction errors as the binary BitNode (better residual connections, error-symmetric architecture) 3. Accept the $7\times$ gap from fast_ce and close it with a few sequential fine-tuning iterations (hybrid approach) 4. Find a completely different $O(\log N)$ training paradigm (learned BP, transformer-based)

10. Code and Reproduction

All code is available at: `/Users/ytnspybq/PycharmProjects/polar_codes_MAC/to_git_v2/`

Key files: - `neural/npd_pytorch.py` — Working single-user NPD port (verified BLER=0.000 on test) - `neural/poc_two_phase_v2.py` — Two-phase iterative refinement - Inline scripts in this session — WHT decoder, 4-class fast_ce, noisy teacher forcing

11. Summary of All Results

Approach	Architecture	BLER at N=32	vs SC (0.046)	Training Depth
Sequential tree walk (d=16)	39K, tree walk	0.046	1.0×	$O(N \log N)$
Sequential tree walk (d=32)	153K, tree walk	0.037	0.8×	$O(N \log N)$
WHT fast_ce, shared, binary	6K, WHT	0.606	13.2×	$O(\log N)$
WHT fast_ce, shared, character	23K, WHT	0.336	7.3×	$O(\log N)$
WHT fast_ce, per-channel, large	283K, WHT	0.254	5.5×	$O(\log N)$
Direct 4-class fast_ce	26K, original	0.340	7.4×	$O(\log N)$
WHT + noisy TF (10%)	5K, WHT	0.378	8.2×	$O(\log N)$
WHT + noisy TF (20%)	5K, WHT	0.414	9.0×	$O(\log N)$
Two-phase iterative (0 refine)	63K, 3×NPD	0.948	20.6×	$O(\log N)$
Two-phase iterative (2 refine)	63K, 3×NPD	0.518	11.3×	$O(\log N)$

12. References

1. Aharoni, Huleihel, Pfister, Permuter — “Data-Driven Neural Polar Codes for Unknown Channels With and Without Memory,” IEEE ISIT 2024
2. Sasoglu, Abbe, Telatar — “Polar Codes for the Two-User Multiple-Access Channel,” IEEE TIT 2013
3. Ren, Bhatt, Mondelli — “SC Decoding for General Monotone Chain Polar Codes,” arXiv 2025
4. Hebbbar et al. — “CRISP: Curriculum Based Sequential Neural Decoders,” ICML 2023
5. Duckworth et al. — “Parallel Scheduled Sampling,” arXiv:1906.04331, 2019
6. Nachmani et al. — “Deep Learning Methods for Improved Decoding of Linear Codes,” IEEE JSTSP 2018